TALLER PRÁCTICO — DESARROLLO DE APLICACIONES MÓVILES

Datos Abiertos de Colombia (Flutter)

Autor:	Samuel Rincón (UCEVA)
Repositorio Público:	https://github.com/Sarincon508/taller_datos_abiertos
Ramas GitFlow:	**main** (producción) • **dev** (integración) • **feature/taller_api_colombia**
API Oficial:	API Colombia v1 (https://api-colombia.com/)
Paquetes Clave:	**http** (^1.6.0) • **go_router** (^18.1.1) • **flutter_dotenv** (^6.0.1) • **intl** (^0.20.3)

1. Endpoints de API Colombia Seleccionados (Mínimo 4)

Se implementó consumo completo en dos niveles (Listado general y Detalle por ID) con decodificación UTF-8 para garantizar soporte de caracteres especiales y acentos:

Módulo / Recurso	Ruta Listado (GET)	Ruta Detalle (GET)	Información Extraída y Atributos
1. Departamentos	`/api/v1/Department`	`/api/v1/Department/{id}`	32 departamentos, capital, población, superficie (km²), municipios y prefijo telefónico.
2. Atracciones Turísticas	`/api/v1/TouristicAttraction`	`/api/v1/TouristicAttraction/{id}`	Parques naturales, monumentos, coordenadas lat/lon, galería fotográfica y ciudad.
3. Presidentes	`/api/v1/President`	`/api/v1/President/{id}`	Mandatarios de la República, periodos, partido político, retrato oficial y biografía.
4. Platos Típicos	`/api/v1/TypicalDish`	`/api/v1/TypicalDish/{id}`	Gastronomía regional, fotografía, desglose de ingredientes y departamento de origen.

2. Arquitectura y Estructura por Capas

La aplicación adopta una arquitectura modular limpia con separación estricta de responsabilidades:

Capa	Ubicación	Responsabilidad Técnica
Config	`lib/config/`	Carga de variables de entorno (.env) mediante Environment con fallbacks y constantes.
Models	`lib/models/`	Entidades fuertemente tipadas con serialización fromJson y toJson para cada endpoint.
Services	`lib/services/`	Cliente ApiColombiaService con manejo de ApiException, status codes (200, 404, 500) y timeouts.
Routes	`lib/routes/`	Configuración declarativa de go_router con subrutas, nombres de ruta y paso de parámetros.
Themes	`lib/themes/`	Temas Material 3 claro y oscuro con identidad tricolor colombiana (azul, oro, esmeralda, carmesí).
Views	`lib/views/`	Pantallas principales: Dashboard interactivo, 4 Listados con búsqueda en vivo y 4 Detalles completos.
Widgets	`lib/widgets/`	Componentes reutilizables: LoadingView, ErrorView, EmptyView, CustomSearchBar, etc.

3. Navegación Maestro-Detalle con go_router

Todas las rutas están declaradas con nombre y path jerárquico. El paso de parámetros soporta tanto pathParameters (ID de registro) como paso instantáneo mediante extra, permitiendo también deep-linking directo:

Nombre de Ruta	Path	Parámetros	Pantalla Destino
`dashboard`	`/`	Ninguno	DashboardView
`departments`	`/departments`	Ninguno	DepartmentsListView
`department-detail`	`/departments/:id`	:id (int), extra: Department?	DepartmentDetailView
`attractions`	`/attractions`	Ninguno	AttractionsListView
`attraction-detail`	`/attractions/:id`	:id (int), extra: TouristicAttraction?	AttractionDetailView
`presidents`	`/presidents`	Ninguno	PresidentsListView
`president-detail`	`/presidents/:id`	:id (int), extra: President?	PresidentDetailView
`typical-dishes`	`/typical-dishes`	Ninguno	TypicalDishesListView
`typical-dish-detail`	`/typical-dishes/:id`	:id (int), extra: TypicalDish?	TypicalDishDetailView

4. Manejo de Estados y Flujo GitFlow

Manejo de Estados: Cada módulo implementa 4 estados visuales claros: (1) **Cargando** (spinner animado con mensaje contextual), (2) **Éxito** (ListView.builder con pull-to-refresh), (3) **Error** (código HTTP 404/500/red con botón Reintentar), y (4) **Búsqueda vacía**.

Flujo GitFlow: El repositorio cuenta con la estructura formal de ramas: **main** (versión estable), **dev** (integración) y **feature/taller_api_colombia**. Se realizaron 9 commits atómicos con la convención Conventional Commits (feat, test, docs) e integración mediante merge commits --no-ff que simulan el Pull Request.